

A
Project Report On

Form Extraction

BTech-IT, Sem VI

Prepared By:
Suryadeepsinh Gohil (IT-037)
Parth Kacha (IT-048)

Guided By:
Prof. Vipul K Dabhi (HOD)
Prof. Harshad. B. Prajapati
Dept. of Information Technology



Department of Information Technology
Faculty of technology,
Dharmsinh Desai University
College road, Nadiad- 387001

October, 2026

CANDIDATE'S DECLARATION

We declare that the 6th semester report entitled **Form Extraction** is our own work conducted under the supervision of the guide Prof. Vipul K Dabhi (HOD) & Prof. Harshad B. Prajapati.

We further declare that to the best of our knowledge the report for B.Tech. VI semester does not contain part of the work which has been submitted either in this or any other university without proper citation.

Candidate's Signature
Suryadeepsinh Gohil
Student ID: 23ITUOS037

Candidate's Signature
Parth Kacha
Student ID: 23ITUBS048

DHARMSINH DESAI UNIVERSITY
NADIAD-387001, GUJARAT



CERTIFICATE

This is to certify that the project carried out in the subject of Project-II, entitled Form Extraction and recorded in this report is a bonafide report of work of

1) Suryadeepsinh Gohil Roll No. IT037 ID No: 23ITUOS037

**2) Parth Kacha Roll No. IT048 ID No: 23ITUBS048
of Department of Information Technology, semester VII . were involved in Project work
during academic year 2026-2027.**

Prof. (Dr.) V. K. Dabhi
(Project Guide),
Head , Department of Information Technology,
Faculty of Technology,
Dharmsinh Desai University, Nadiad
Date:

Prof. (Dr.) H. B. Prajapati
(Project Guide),
Department of Information Technology,
Faculty of Technology,
Dharmsinh Desai University, Nadiad
Date:

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to the Department of Information Technology, Dharmsinh Desai University, for providing us with the opportunity to undertake this project titled “*Form Extraction: Automated Data Extraction System*”.

We are deeply thankful to our respected guide, Prof. (Dr.) V. K. Dabhi & Prof. (Dr.) H. B. Prajapati, for their valuable guidance, constant encouragement, and insightful suggestions throughout the development of this project. Their support helped us in understanding the concepts and completing the project successfully.

We would also like to extend our appreciation to all the faculty members of the Department of Information Technology for their continuous support and for providing the necessary resources required for the completion of this project.

Finally, we express our heartfelt thanks to our teammates for their cooperation, dedication, and teamwork, which played a crucial role in the successful completion of this project.

Suryadeepsinh Gohil
Parth Kacha

Table Of Content

<u>Title</u>	<u>Page No</u>
<u>ABSTRACT</u>	<u>6</u>
<u>CHAPTER 1: INTRODUCTION</u>	<u>7</u>
<u>CHAPTER 2: PROJECT MANAGEMENT</u>	<u>8</u>
<u>CHAPTER 3: SYSTEM REQUIREMENTS STUDY</u>	<u>9</u>
<u>CHAPTER 4: SYSTEM ANALYSIS</u>	<u>11</u>
<u>CHAPTER 5: SYSTEM DESIGN</u>	<u>12</u>
<u>CHAPTER 6: IMPLEMENTATION</u>	<u>14</u>
<u>CHAPTER 7: TESTING</u>	<u>15</u>
<u>CHAPTER 8: USER MANUAL</u>	<u>16</u>
<u>CHAPTER 9: LIMITATIONS AND FUTURE ENHANCEMENTS</u>	<u>17</u>
<u>CHAPTER 10: CONCLUSION AND DISCUSSION</u>	<u>19</u>
<u>APPENDICES</u>	
<u>REFERENCES</u>	

ABSTRACT

Extracting structured data—such as key-value pairs, checkboxes, and complex table grids—from scanned or photographed forms is a notoriously difficult problem in the field of document digitization. Historically, solutions have fallen into two extreme categories: brittle, rule-based Zonal OCR that breaks if a document is shifted by a mere millimeter, or excessively complex generative Large Language Models (LLMs) that are slow, expensive, and highly prone to hallucinating data. This project presents a high-performance, deterministic Document Extraction Engine that perfectly bridges this gap.

By utilizing advanced Computer Vision (OpenCV) algorithms alongside state-of-the-art Optical Character Recognition (PaddleOCR), the system extracts data with extreme geometric precision. The core innovation of this engine is its Template-Based Homography Alignment pipeline. A user first registers a blank "Template" form, allowing the system to automatically analyze the structural graph of the page (detecting horizontal lines, vertical lines, and bounding boxes). Once established, any filled version of that form uploaded by a user is mathematically warped, rotated, and geometrically aligned to the master template using feature matching (ORB/SIFT).

Because the incoming document is corrected to perfectly match the template's coordinate system, the engine achieves lightning-fast, zero-shot structured extraction. No LLM parsing is required, resulting in 100% deterministic outputs, rapid processing speeds, and native structural support for complex form elements like checked/unchecked boolean boxes and unstructured tables.

CHAPTER 1: INTRODUCTION

1.1 Background

The transition from physical paper trails to digital databases remains a significant bottleneck for enterprises, healthcare facilities, and government organizations. Manual data entry is inherently slow and fraught with human error. While automated extraction systems have existed for decades, modern business workflows heavily rely on mobile document capture. This introduces extreme visual distortions: perspective warping, variable lighting, severe skew angles, and differing focal lengths. Traditional zonal OCR systems simply cannot process these mobile uploads without manual intervention, prompting a need for a highly resilient, automated alignment system.

1.2 Objective

The primary objective of this project is to architect, develop, and deploy a robust, highly accurate, and deterministic form extraction API and corresponding web dashboard. The engine explicitly avoids generative AI for structural mapping to ensure that data is never hallucinated. The system must operate efficiently on standard hardware, provide a seamless user interface, and handle asynchronous batch processing for large-scale document loads.

1.3 Scope

The scope of this project encompasses both the backend processing engine and the frontend user dashboard. On the backend, the system must process raw images, execute orientation correction, perform perspective deskewing, register form templates, and extract fields. It must support multiple input types: handwritten text, printed text, checkboxes, and bordered/borderless tables. On the frontend, it must provide secure authentication, template management interfaces, extraction result viewing, and file upload mechanisms.

CHAPTER 2: PROJECT MANAGEMENT

2.1 Methodologies

This project was developed utilizing Agile software development methodologies. The system was separated into modular, self-contained epics: Backend API architecture, Computer Vision pipeline (deskew and homography), OCR text extraction, Database integration, and Frontend UI/UX development. This iterative approach allowed for continuous testing of the extraction accuracy as the feature set expanded.

2.2 Tools and Version Control

Development was managed using Git for version control, allowing seamless branching and rollback of experimental Computer Vision algorithms. Code editing and debugging were performed in Visual Studio Code. Package management was handled strictly via Python virtual environments (pip) for the backend and Node Package Manager (npm) for the frontend build tools.

CHAPTER 3: SYSTEM REQUIREMENTS STUDY

3.1 Hardware Requirements

Due to the use of heavy Optical Character Recognition models and matrix transformations, the system requires moderate hardware to run efficiently. A minimum of 8GB of RAM is recommended. The backend utilizes a CPU with AVX support for optimal PaddlePaddle inference, though GPU acceleration can be enabled for massive batch processing.

3.2 Software Requirements

The backend requires Python 3.10 or higher. The frontend requires Node.js 18+ and relies on the Vite build system. The system can run natively on Windows, macOS, and Linux.

3.3 Required Technologies

Frontend Architecture:

React & Vite: Utilized for highly responsive single-page application rendering and instant Hot Module Replacement during development.

TailwindCSS: A utility-first CSS framework used to rapidly build a clean, modern, and fully responsive user dashboard.

Backend Architecture:

FastAPI & Python: Chosen for its high-performance asynchronous request handling, automatic OpenAPI documentation, and strict Pydantic type validation.

Computer Vision & Machine Learning:

OpenCV-Python (cv2): The backbone of the geometric alignment engine. Handles Canny edge detection, Hough Line Transforms, contour analysis, and RANSAC homography.

PaddleOCR & Numpy: PaddleOCR is leveraged for its unparalleled accuracy on multilingual, handwritten, and variable-font texts. Numpy handles the high-speed matrix calculations for intersection-over-union mapping.

Infrastructure:

Storage Engine: Supports SQLite and Local File Storage for immediate local development, with abstract integration for PostgreSQL and AWS S3 via Boto3 for cloud production deployment.

Authentication: Clerk (@clerk/clerk-react) provides enterprise-grade, JWT-based secure sessions across the decoupled stack.

CHAPTER 4: SYSTEM ANALYSIS

4.1 Existing Systems vs. Proposed System

Existing document parsing systems generally rely on one of two paradigms, both of which possess critical flaws for this specific use case.

1. Zonal OCR Systems:

These legacy systems require operators to draw hardcoded boxes over a document. If a scanned document is physically rotated by 3 degrees, or photographed at an angle, the text shifts out of the predefined box. The system extracts garbage data, requiring constant human oversight and manual cropping.

2. LLM-Based Parsing (Generative AI):

Modern solutions pass an image of the document to a Large Language Model (like GPT-4 Vision). While highly flexible, this approach takes 10+ seconds per page, costs significantly per API call, and is prone to "hallucinations"—where the model guesses or fabricates data that isn't actually on the page. Furthermore, LLMs fundamentally misunderstand geometric layouts, meaning checkboxes and complex tables frequently confuse the model.

4.2 The Proposed Solution

The proposed system uses a Homography Alignment engine. Instead of forcing an LLM to guess the structure, the system dynamically calculates the mathematical perspective distortion between the uploaded document and the original blank template. By warping the new document to mathematically match the template, the system completely negates physical shifts and rotations. It then applies highly accurate, deterministic OCR strictly on the known, aligned coordinates. This yields the speed of traditional OCR with the flexibility of modern alignment algorithms, entirely solving the brittleness problem.

CHAPTER 5: SYSTEM DESIGN

5.1 Architectural Decoupling

The system uses a heavily decoupled Client-Server architecture. The React frontend exists purely as a presentation layer, completely stateless. It interacts with the FastAPI backend through RESTful JSON endpoints. The backend itself is stateless (relying on the database and S3 for persistence), allowing it to be horizontally scaled behind a load balancer. Code is organized into strict domains: API routers (`template_router.py`, `filled_form_router.py`), and business logic services (`ocr_service.py`, `preprocess_service.py`).

5.2 End-to-End Workflow

Phase 1: Template Registration (How a template is created)

1. Upload: The user uploads a pristine, blank (unfilled) form image through the dashboard.
2. Preprocessing: The backend `preprocess_service` analyzes the image, automatically corrects the orientation (e.g., if it is upside down), and deskews it to a perfectly upright position.
3. Structural Analysis: OpenCV algorithms process the image. Canny edge detection isolates contrasting lines. Hough Line Transforms identify all horizontal and vertical bounding lines. Contour analysis detects squares (checkboxes) and grids (tables).
4. Persistence: This geometric map is saved as the master Template graph to the database, alongside the processed master image in S3/Local Storage.

Phase 2: Filled Form Extraction (How data is extracted)

1. Upload: The user uploads a filled version of the form (which may be photographed at an angle) and specifies the corresponding Template ID.
2. Feature Matching: The Backend loads both the filled image and the master template image. Algorithms (like ORB or SIFT) detect hundreds of

visual keypoints on both images (e.g., the corners of logos, printed instructional text).

3. Homography Warping: RANSAC is used to filter out mismatched keypoints (like the user's handwriting) and calculate a homography matrix. The filled image is geometrically warped to exactly overlay the master template.

4. OCR & Mapping: PaddleOCR extracts every text bounding box from the newly warped image. The system mathematically calculates the Intersection-Over-Union (IoU) of these text boxes against the known template fields. Text falling inside a template region is assigned as the value for that field.

5. Delivery: The highly structured JSON data is instantly returned to the frontend.

CHAPTER 6: IMPLEMENTATION

The implementation strictly divides API logic from core computational services to maintain clean code architecture. `main.py` serves exclusively as the ASGI entry point, loading environment variables, mounting the CORS middleware, and registering the API routers. The heavy lifting resides in dedicated Python modules:

template_service.py: Orchestrates the OpenCV homography alignment and field mapping.

ocr_service.py: Wraps the PaddleOCR inference engine, formatting raw neural network outputs into standardized bounding box dictionaries.

blob_store.py: Provides an agnostic storage abstraction interface, allowing the system to hot-swap between local disk saves and AWS S3 uploads without touching business logic.

On the frontend, the React UI utilizes modern React Hooks (`useState`, `useEffect`, `useCallback`) for state management and API communication, utilizing Clerk for route guarding and session handling.

CHAPTER 7: TESTING

Testing is a critical phase for a Computer Vision pipeline. The homography engine was tested against heavily distorted images—documents photographed from extreme angles, under poor lighting conditions, and with significant background noise (e.g., a form lying on a cluttered wooden desk). The RANSAC outlier rejection algorithm successfully ignored the background clutter, locking onto the form's printed text to achieve sub-pixel alignment accuracy. PaddleOCR's robustness was rigorously tested against messy, cursive handwriting, proving highly resilient compared to basic Tesseract implementations.

CHAPTER 8: USER MANUAL

To operate the Form Extraction Engine locally:

1. Environment Setup: Ensure the `.env` files in both the root directory and the `/ui` directory contain valid Clerk API keys and local storage paths.
2. Start Backend: Open a terminal in the root directory, activate the Python virtual environment, and execute: `uvicorn main:app --reload`.
3. Start Frontend: Open a terminal in the `/ui` directory and execute: `npm run dev`.
4. Login: Navigate to `http://localhost:5173` in a web browser and authenticate using the Clerk portal.
5. Template Registration: Navigate to the "Templates" tab. Upload a blank, completely empty version of a form. Give it a descriptive name.
6. Data Extraction: Click on the newly created template. Upload filled versions of that form. The engine will process the alignment and display the extracted key-value pairs instantly.

CHAPTER 9: LIMITATIONS AND FUTURE ENHANCEMENTS

9.1 Limitations

The homography approach relies fundamentally on structural and visual similarity between the template and the filled form. Extreme physical crumpling of the paper, or scenarios where the document's borders are completely obscured and no printed anchor text is visible, may cause the feature matching algorithm to fail. Additionally, highly complex, nested tables with invisible borders remain challenging for the heuristic line detectors.

9.2 Future Enhancements

The architecture is highly extensible. The following features are primed for future development:

Advanced Verification UI (Human-in-the-Loop): Build a side-by-side verification interface. When users review the extracted JSON, they can click on a specific key-value pair and the UI will pan/zoom to a highlighted crop of that exact field on the original document. This massively speeds up human QA workflows.

Multi-Page Document Packets: Extend the engine to support 10+ page PDF uploads. The system will automatically run classification on each page, identify which template it belongs to, and extract the entire packet seamlessly.

Anchor-Based Local Alignment: Instead of warping the entire page, the system will identify fixed local anchors (like barcodes or specific logos) and calculate localized offsets. This will provide massive resilience to extremely crumpled or folded paper documents where global homography fails.

Export Integrations: Introduce one-click export functionality to CSV, Excel, Google Sheets, or webhooks, allowing seamless data piping directly into downstream ERP or CRM systems.

API Key Management: Allow users to generate API keys directly from the dashboard to programmatically trigger batch extractions from their own backend systems.

CHAPTER 10: CONCLUSION AND DISCUSSION

By shifting from a trendy but brittle LLM-based architecture to a strictly deterministic Homography + OCR pipeline, this project achieved a massive leap in processing speed and reliability. The system guarantees zero hallucination rates and provides native, highly robust support for complex elements like checkboxes and tables that consistently confuse generative models. This project serves as a compelling proof-of-concept that targeted, traditional Computer Vision techniques often vastly outperform generalized AI models for structured, repetitive tasks in enterprise environments.

APPENDICES

Appendix A: Backend API Routers

The backend API features endpoints such as `/register-template`, `/extract-filled`, and `/filled-forms` which handle multipart/form-data uploads and return structured JSON responses.

REFERENCES

1. OpenCV Documentation (Homography & Feature Matching):
<https://docs.opencv.org/>
2. PaddleOCR Repository & Documentation:
<https://github.com/PaddlePaddle/PaddleOCR>
3. FastAPI Documentation: <https://fastapi.tiangolo.com/>
4. Clerk React Authentication SDK: <https://clerk.com/docs>